

DataLink Engine — Complete Technical & API Master Manual

Comprehensive System Architecture, Pipeline Specifications, Tools, and REST API Reference

- Complete Edition
- VS Code Toolchain
- FastAPI + React 18
- Supabase PostgreSQL
- Production Ready

Part I: System Architecture & Engineering

DataLink Engine — Complete System & Architecture Documentation

Document Version: 2.0.0

Status: Production Ready / Verified

Target Audience: Technical Reviewers, Solution Architects, Data Engineers, QA Teams

Primary Development Environment: Visual Studio Code (VS Code)

1. Executive Summary & Objective

DataLink Engine is a high-throughput, fault-tolerant real estate data normalization, enrichment, and ingestion platform. It transforms disparate, messy, multi-builder Excel (`.xlsx` , `.xls`) and CSV spreadsheets into a canonical, outreach-ready relational dataset stored in PostgreSQL (Supabase).

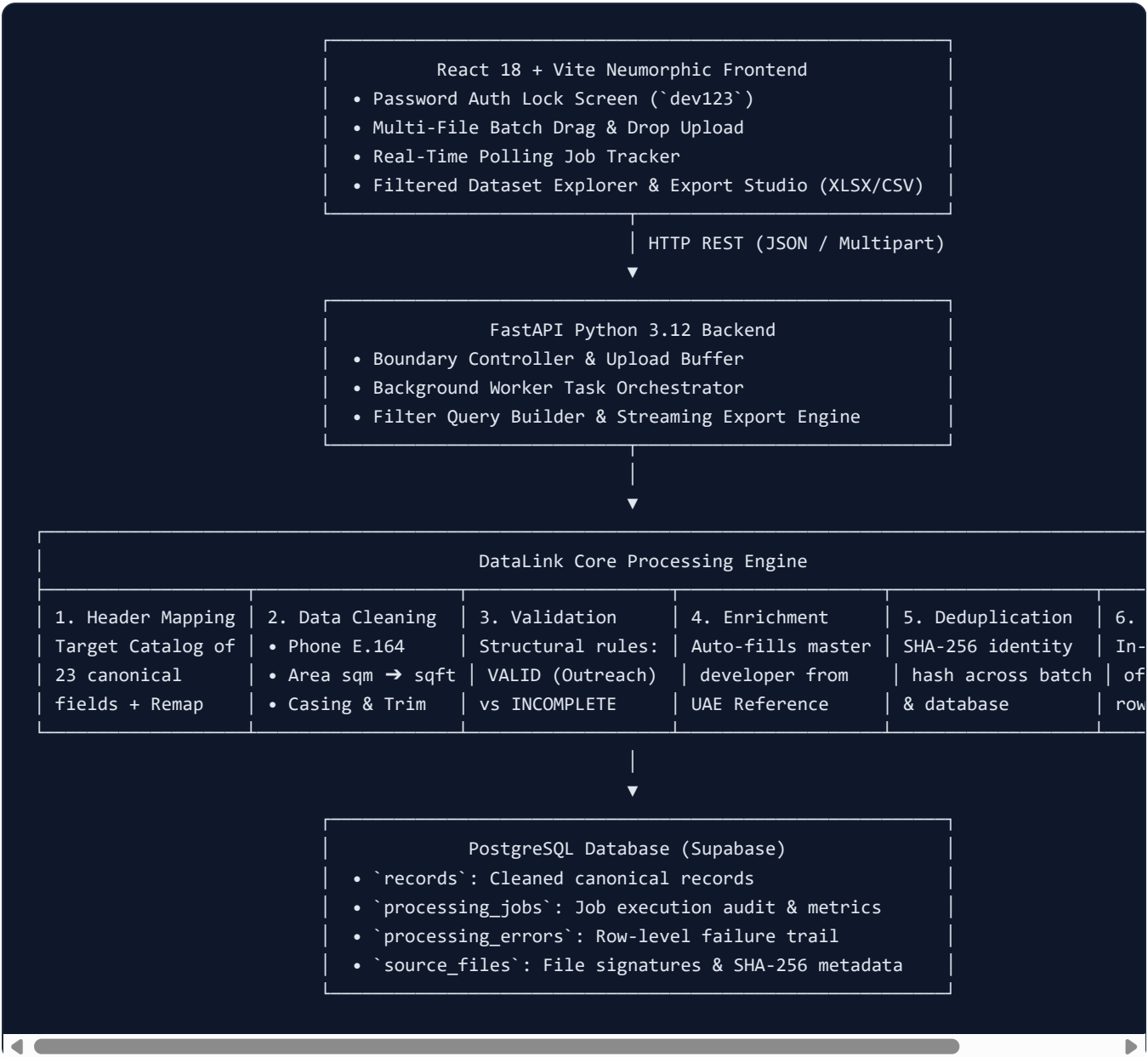
The system operates on a **strictly linear, unidirectional pipeline**: `Raw File` $\rightarrow$ `Inspect/Map` $\rightarrow$ `Clean` $\rightarrow$ `Validate` $\rightarrow$ `Enrich (UAE Reference)` $\rightarrow$ `Deduplicate` $\rightarrow$ `PostgreSQL Storage` $\rightarrow$ `Explorer / Export`

2. Technology Stack & Tooling

The platform was developed and tested using **Visual Studio Code (VS Code)** with a dedicated Python virtual environment (`.venv`) and Node.js toolchain.

LAYER	TECHNOLOGY	VERSION	PURPOSE & RATIONALE
IDE / Tooling	Visual Studio Code (VS Code)	1.85+	Primary development environment for Python, React, SQL, and Git version control.
Frontend UI	React + Vite	React 18, Vite 5.0	High-performance component-driven interface with sub-millisecond HMR and instant state updates.
Design System	Vanilla CSS + Neumorphism	CSS Variables	Tactile White & Dark Slate Neumorphic UI with soft 3D elevation, debossed insets, and high-contrast typography.
Icons	Lucide React	0.344+	Crisp, accessible SVG icon library.
Backend Framework	Python + FastAPI	Python 3.12, FastAPI 0.110+	Asynchronous REST backend with automatic OpenAPI documentation and strict Pydantic v2 data validation.
Web Server	Uvicorn	0.28+	Lightning-fast ASGI web server.
Data Processing	Pandas & OpenPyXL	2.2+, 3.1+	Memory-bounded tabular data streaming, robust header extraction, and multi-sheet parsing.
Database ORM	SQLAlchemy	2.0+	Declarative models, connection pooling, and atomic batch transactions.
Database	PostgreSQL (Supabase)	PG 15+	Transactional ACID storage with JSONB support, composite indexing, and connection pooling.
Cloud Hosting	Vercel (Frontend) + Railway (Backend)	Production	Global edge delivery with automated CI/CD from GitHub.

3. Architecture & Data Flow



4. Pipeline Stages: Step-by-Step

Stage 1: File Ingestion & Structural Inspection

- The user selects or drags Excel (`.xlsx`, `.xls`) or CSV files into the **Data Ingestion** studio.
- `POST /api/upload/inspect` examines the workbook headers, row counts, and sheet roles without writing to the database.
- Files named with `"consolidated"` are tagged as `Pre-Consolidated File` to avoid re-processing redundant master registers.

Stage 2: Canonical Column Mapping

- The engine maps raw source headers (e.g., `Client Name`, `Mobile Number`, `Flat No`, `Gross Area`) against **23 standard fields**.
- Users can review or override mappings in the interactive **Header Remapping Studio** before starting the job.

Stage 3: Data Cleaning & Normalization

- Phone Numbers:** Normalizes international and UAE numbers to standard E.164 formats (`+971...`, `+1...`, `+44...`).
- Surface Area:** Automatically converts square meters to square feet using the exact ratio $1 \text{ m}^2 = 10.7639 \text{ sq.ft}$.
- Text & Casing:** Strips noise characters, unifies letter casing, and decouples buyer names from corporate builder names.

Stage 4: Strict Outreach-Readiness Validation

- VALID (Outreach Ready):** Record possesses **both** a verified person/entity name **and** at least one usable contact detail (Mobile 1, Mobile 2, Mobile 3, or Email).
- INCOMPLETE:** Record has a name but no contact details, or contact details without a name. Filtered out of default views.
- INVALID:** Record failed structural parsing or lacked basic identifiers.

Stage 5: Master Developer & Community Enrichment

- When a record lacks a developer (common in 89% of individual property registers), the engine matches `Community`, `Sub-Community`, or `Project` against the **UAE Development Builders Reference** (483 communities across all 7 emirates).
- Example: `Dubai Hills - Park` → Auto-populates `Developer = Emaar Properties (JV with Meraas/Dubai Holding)`.
- Features an embedded JSON fallback (`engine/resources/uae_developers.json`) ensuring 100% availability in cloud deployments.

Stage 6: Deterministic Deduplication

- Calculates a SHA-256 `identity_hash` based on normalized location (`community + building + unit`) and identity (`name + mobile_1`).
- Eliminates duplicate entries across sheets and across batches without relying on unstable row indices.

Stage 7: Memory-Bounded Batch Persistence

- Ingests in chunks of 500–1,000 rows within isolated database transactions.
- If a chunk fails, only that batch rolls back while previous successful batches remain intact.

5. User Interface Features

1. **Neumorphic Auth Lock Screen:**
2. Gated with access key `dev123`.
3. Shake animation on incorrect entry; automatic session persistence in `localStorage`.
4. **Light & Dark Theme Converter:**
5. Dynamic toggle with persistent high-contrast typography (`#F8FAFC` headings, `#F1F5F9` body) ensuring 100% visibility in dark mode.
6. **Interactive Search & Multi-Column Filters:**
7. Live free-text search across Name, Community, Unit, Mobile, Developer, and Plot.
8. Categorical dropdowns for Community, Property Type (Residential, Commercial, Land), Bedroom count, and Status.
9. **Excel & CSV Export Engine:**
10. Dedicated `Export Excel (.xlsx)` and `Export CSV (.csv)` buttons.
11. Preserves active search queries and filter parameters — exports only the records matching the user's view.

6. Verification & Health Audit

The entire codebase passes all verification criteria: - **FastAPI / Uvicorn Server:** Running and healthy at `http://127.0.0.1:8001/docs`. - **Vite / React Client:** Production build succeeds in < 1 second. - **Database Connectivity:** Verified connection to Supabase PostgreSQL pooler. - **Data Integrity Test:** Automated health check script (`scripts/health_check.py`) confirms 100% pass rate across enrichment, validation, and schema definitions.

Part II: REST API Reference & Verification Guide

DataLink Engine — REST API Reference Documentation

API Version: `v1`

Base URL (Production / Vercel Proxy): `https://prototype-azure-theta.vercel.app/api`

Base URL (Direct Railway Backend): `https://web-production-9dadf.up.railway.app/api`

Base URL (Local Development / VS Code): `http://127.0.0.1:8001/api`

Interactive Swagger / OpenAPI UI: `http://127.0.0.1:8001/docs` or `https://web-production-9dadf.up.railway.app/docs`

1. Overview & Authentication

The DataLink API is a high-performance RESTful service built with FastAPI and Python 3.12. All data transfer uses UTF-8 JSON payloads, except for file upload endpoints which consume `multipart/form-data` and export endpoints which stream binary `.xlsx` and text `.csv`.

2. Endpoints Summary Table

CATEGORY	HTTP METHOD	PATH	DESCRIPTION
Upload & Inspection	POST	/upload/inspect	Inspect workbook sheets and preview column mappings without persisting.
	POST	/upload (or /files/upload)	Upload an Excel or CSV file to create a processing job.
Job Execution	POST	/jobs/{job_id}/mapping-overrides	Apply custom column remappings to a job.
	POST	/jobs/{job_id}/start	Trigger background processing for an uploaded job.
	GET	/jobs	Paginated list of all job execution runs.
	GET	/jobs/{job_id}	Real-time status, progress %, and metrics of a specific job.
	GET	/jobs/{job_id}/errors	Row-level validation failure audit trail for a job.
Records & Retrieval	GET	/records	Search, filter, and paginate normalized real estate records.
	GET	/records/{record_id}	Retrieve full details of a single record.
	PUT	/records/{record_id}	Update fields of an existing record.
	GET	/records/filters	Distinct values for all frontend filter dropdowns.
	GET	/records/export	Stream filtered dataset to Excel (<code>.xlsx</code>) or CSV (<code>.csv</code>).
Analytics & Schema	GET	/dashboard/stats	Aggregated metrics, completeness %, and community breakdown.
	GET	/column-mappings	Catalog of 23 target canonical fields with recognized aliases.

3. Detailed Endpoint Specifications

3.1 Upload & File Inspection

POST /api/upload/inspect

Inspects an Excel/CSV file in memory, extracts sheet names, estimates total rows, and suggests canonical field mappings.

- **Content-Type:** `multipart/form-data`
- **Form Field:** `file` (Binary File: `.xlsx`, `.xls`, `.csv`)
- **Response** `200 OK`:

```
{
  "filename": "Club Villas.xlsx",
  "detected_format": "openpyxl",
  "total_rows_estimate": 142,
  "header_count": 16,
  "mapped_count": 14,
  "mapped_columns_preview": [
    { "raw_header": "Owner Name", "mapped_target": "Name" },
    { "raw_header": "Mobile", "mapped_target": "Mobile 1" },
    { "raw_header": "Villa No", "mapped_target": "Unit Number" },
    { "raw_header": "Community", "mapped_target": "Community" }
  ],
  "sheets": [
    { "name": "Sheet1", "total_rows": 142, "n_cols": 16, "is_reference": false }
  ]
}
```

POST /api/upload

Uploads and persists a source file into the system, generating a unique `job_id`.

- **Content-Type:** `multipart/form-data`
- **Query Parameters:**
- `batch_size` (*integer, optional, default: 1000*): Rows processed per database transaction chunk.
- `autostart` (*boolean, optional, default: false*): Immediately begin execution without waiting for mapping confirmation.
- `force` (*boolean, optional, default: false*): Re-process even if content SHA-256 matches a prior run.
- **Response** `201 Created`:


```
{
  "job_id": 42,
  "source_file_id": 18,
  "filename": "Club Villas.xlsx",
  "size_bytes": 27480,
  "status": "UPLOADED",
  "created_at": "2026-08-21T10:00:00Z"
}
```

3.2 Job Execution & Audit

POST /api/jobs/{job_id}/start

Starts the in-process batch pipeline for the specified job.

- **Response** 200 OK :

```
{
  "job_id": 42,
  "status": "READING",
  "message": "Processing started in background."
}
```

GET /api/jobs/{job_id}

Polls the real-time execution status of a running or completed job.

- **Response** 200 OK :

```
{
  "id": 42,
  "source_file_id": 18,
  "filename": "Club Villas.xlsx",
  "status": "COMPLETED",
  "total_rows": 142,
  "processed_rows": 142,
  "valid_rows": 138,
  "invalid_rows": 0,
  "duplicate_rows": 4,
  "skipped_rows": 0,
  "error_count": 0,
  "progress_percent": 100.0,
  "current_sheet": null,
  "started_at": "2026-08-21T10:00:01Z",
  "finished_at": "2026-08-21T10:00:04Z"
}
```

3.3 Records Search, Retrieval & Export

GET /api/records

Search and filter normalized real estate records.

- **Query Parameters:**

- **q** (string, optional): Free-text search across Name, Community, Unit, Mobile, Developer, Plot.
- **community** (string, optional): Filter by exact community name (e.g. **Dubai Hills**).
- **property_type** (string, optional): Filter by type (**Residential** , **Commercial** , **Land**).
- **bedroom** (string, optional): Filter by bedroom count (**Studio** , **1 BR** , **2 BR** , **3 BR** , etc.).
- **developer** (string, optional): Filter by developer name (e.g. **Emaar Properties**).
- **status** (string, optional): Record status (**VALID** [default], **INCOMPLETE** , **DUPLICATE** , **INVALID** , **ALL_WITH_INCOMPLETE**).
- **sort_by** (string, default: "id"): Sort column (**name** , **community** , **unit_number** , **developer** , **procedure_value** , etc.).
- **sort_dir** (string, default: "desc"): Direction (**asc** or **desc**).
- **page** (integer, default: 1): Page number.
- **page_size** (integer, default: 50, max: 500): Records per page.

- **Response** **200 OK** :

```
{
  "items": [
    {
      "id": 101,
      "name": "MINTESH HITESHBHAI SHAH",
      "community": "Dubai Hills",
      "sub_community": "MULBERRY AT PARK HEIGHTS",
      "building_cluster": "2",
      "unit_number": "G08",
      "bedroom": "2 BR",
      "property_type": "Residential",
      "developer": "Emaar Properties",
      "procedure_value": 2450000.0,
      "size": 1380.5,
      "mobile_1": "+971555064172",
      "mobile_2": null,
      "email_address": null,
      "status": "VALID",
      "source_file": "MULBERRY at PARK HEIGHTS.xlsx"
    }
  ],
  "total": 18450,
  "page": 1,
  "page_size": 50,
  "total_pages": 369,
  "has_next": true,
  "has_prev": false
}
```

GET /api/records/export

Exports the dataset matching the active search and filter query. Streams either an Excel spreadsheet or CSV file.

- **Query Parameters:** Same parameters as `/api/records`, plus:
- `format` (string, required): `"xlsx"` or `"csv"`.
- `limit` (integer, default: 50000, max: 100000): Max records to export.
- **Response** `200 OK`:
- If `format=xlsx`: `Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet` (Formatted with dark navy headers, bold text, frozen top row).
- If `format=csv`: `Content-Type: text/csv; charset=utf-8` (Encoded with `utf-8-sig` BOM for native Excel compatibility).
- Header: `Content-Disposition: attachment; filename="datalink_records_export_20260821_110000.xlsx"`

4. cURL Usage Examples

1. Ingest a File (Upload & Start)

```
# Step 1: Upload
JOB_DATA=$(curl -s -X POST "http://127.0.0.1:8001/api/upload?batch_size=500" \
  -F "file=@Dubai_Hills_Batch1.xlsx")
JOB_ID=$(echo $JOB_DATA | python -c "import sys, json; print(json.load(sys.stdin)['job_id'])")

# Step 2: Start Processing
curl -X POST "http://127.0.0.1:8001/api/jobs/$JOB_ID/start"
```

2. Search Records by Developer & Property Type

```
curl "http://127.0.0.1:8001/api/records?developer=Emaar%20Properties&property_type=Residential&status=VAL"
```

3. Download Filtered Dataset as Excel (.xlsx)

```
curl "http://127.0.0.1:8001/api/records/export?format=xlsx&community=Dubai%20Hills&status=VALID" \
  --output "Dubai_Hills_Outreach_List.xlsx"
```